Alocação Sequencial - Pilhas

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Pilha

- Conceito intuitivo: pilha de pratos
  - Inserções de novos pratos no topo
  - Remoções de pratos do topo (até atingir o prato que deseja-se remover)
- Definição trivial:
  - LIFO: Last In First Out
- Exemplos de programas que utilizam pilhas:
  - Editores de texto, planilhas, browsers, etc
    - ícone de restauração de texto antes da última modificação
    - modificações armazenadas na pilha para garantir a sequência de retorno

Operações sobre uma pilha

- Definida uma pilha P: possíveis operações
  - Empilhar elemento x: push(P,x) - push down
  - Desempilhar elemento: pop(P) - pop up - deve sinalizar se a pilha está vazia
  - iniciar a pilha: init(P)
  - consultar o elemento do topo: top(P) - deve sinalizar se a pilha está vazia
  - verificar se a pilha está vazia: is_empty(P)
  - tamanho da pilha: len(P)

Implementação de pilha em Python

- Uma pilha pode ser facilmente implementada usando lista em Python
  - o topo da pilha é o último elemento
  - a base da pilha o primeiro elemento
  - os métodos append() e pop() podem ser usados para empilhar e desempilhar elementos
  - a operação len(P) é a própria função do Python que opera sobre listas
- Podemos empilhar qualquer tipo de objeto (int, bool, string, tupla, lista, set, etc.)!
- Inclusive podemos misturando tipos diferentes na própria pilha!

Implementação trivial: análoga a de lista!

Implementação de pilha em Python

```
# Inicia uma pilha - vazia
def init(P):
    P = []
# Empilha novo elemento
def push(P, x):
    P.append(x)
# Retorna o elemento do topo da pilha mas não desempilha
def top(P):
    if is_empty(P): return None
    return P[-1]
# Remove elemento do topo da pilha
def pop(P):
    if is_empty(P): return None
    return P.pop()
# retorna True se a pilha está vazia
def is_empty(P):
    return len(P) == 0
```

Implementação de pilha em Python

```
# Empilha e imprime o estado atual da pilha
def Empilha(p, z):
   push(p, z)
   print(p)
# Desempilha e imprime o estado atual da
pilha
def Desempilha(p):
   x = pop(p)
   if x is None: print("Vazia")
   else: print(p)
# inicia a pilha
mp = []
init(mp)
```

```
# operações
Empilha(mp, 1)
Empilha(mp, "tipo de elemento")
Empilha(mp, (5, 4, 3))
Empilha(mp, True)
print(top(mp))
Desempilha(mp)
Desempilha(mp)
Desempilha(mp)
Desempilha(mp)
Desempilha(mp)
print(top(mp))
```

Limitando o tamanho da pilha

- Definir a lista com um número máximo de elementos e limitar o tamanho da pilha é mais eficiente do que inserir cada elemento com append()
  - podemos redefinir as funções acima dessa maneira
  - um parâmetro a mais (MaxTamPilha) nas funções init() e push()
    - push() deve devolver False se a pilha já estiver cheia

Pilha

Exercícios:

1. Reescrever as funções anteriores com a nova definição de pilha com máximo de elementos (sem append()).
2. Pode-se utilizar uma pilha para verificar se os parêntesis dentro de uma expressão estão balanceados (cada abre tem o seu fecha correspondente): a cada abre empilhamos e a cada fecha desempilhamos. Implemente a verificação de balanceamento de parênteses.
3. Supondo agora que a expressão possa conter também [] e {} além de (). Exemplo: **{**a * **[**b + c * **(**d – e**)** * **{**a – b**}]}**

TAD Pilha

- O caso de pilha vazia: caso de erro
  - existem formas de verificar essa condição antes de usar as funções pop() e top()
  - indicar uma exceção quando isso ocorre.
  - a função __len__ assim definida para usar o mesmo nome para um novo objeto.

TAD Pilha

```
class PilhaLista:
    # Pilha como uma lista
    # Construtor da classe PilhaLista
    def __init__(self):
        self._pilha = [] # lista que conterá a pilha

    # retorna o tamanho da pilha
    def __len__ (self):
        return len(self._pilha)

    # retorna True se pilha vazia
    def is_empty(self):
        return len(self._pilha) == 0

    # empilha novo elemento e
    def push(self, e):
        self._pilha.append(e)
```

```
# retorna o elemento do topo da pilha sem retirá-lo
    # exceção se pilha vazia
    def top(self):
        if self.is_empty():
            raise Empty("Pilha vazia")
        return self._pilha[-1]


    # desempilha elemento
    # excessão se pilha vazia
    def pop(self):
        if self.is_empty():
            raise Empty("Pilha vazia")
        return self._pilha.pop()
```

TAD Pilha

Exercício 4: Escreva também dentro do TAD PilhaLista a função __str__(self), para permitir a impressão de uma pilha no formato vertical. Por exemplo, o comando print(P) deve imprimir:

3 - True

2 - (5, 4, 3)

1 - “tipo de elemento”

0 - 1

Aplicações de Pilha

- Pilha: estrutura fundamental em muitas aplicações da computação
- Na execução de um programa, quando se entra num novo bloco, podem existir novas variáveis: são alocadas numa pilha de variáveis (podem existir variáveis com os mesmos nomes em escopos diferentes)
    - a busca sobre qual variável está referenciada é feita do topo para a base da pilha

Aplicações de Pilha

```
def f1():
    a = 1                       # pilha = a, z, a
    b = 1.0                     # pilha = a, z, a, b
    def f2():
        a = 2                   # pilha = a, z, a, b, a
        b = 2.0                 # pilha = a, z, a, b, a, b
        c = 2.0                 # pilha = a, z, a, b, a, b, c
        print(a, b, c, z)
    f2()
    print(a, b, z)              # pilha = a, z, a, b

a = 0                           # pilha = a
z = -1                          # pilha = a, z
f1()
print(a, z)
```

Aplicações de Pilha

- O algoritmo de análise sintática (aquele algoritmo que verifica se a sintaxe do programa está correta - compiladores) usa uma pilha sintática: guarda o contexto do programa sendo analisado.
  - Pilha para controle de símbolos a serem processados, armazenar estados símbolos durante o reconhecimento da gramática e a pilha de execução das funções propriamente dita!
- Durante a execução de um programa: uma função f chama uma função g que chama uma função h
  - Após a execução de h deve-se voltar para g
  - Após a execução de g, deve-se voltar para f
  - os endereços de retorno após a chamada de uma função são colocados numa pilha de execução para que se volte para o lugar certo após a execução das funções
- Caso especial de chamada de funções: funções recursivas
  - Como a função chamada é sempre a mesma, o controle dos endereços de retorno e dos parâmetros de cada chamada é feito pela pilha de execução. Os parâmetros de cada chamada são também empilhados

Notação pós-fixa para funções

- Também chamada de notação polonesa, foi proposta pelo matemático polonês Jan Łukasiewicz (1878 - 1956).
- Apresenta uma série de facilidades em relação a notação tradicional. Nela os operadores aparecem após os operandos e não entre os operandos.
    - Altamente utilizada com propósito de facilitar operações computacionais

Notação pós-fixa para funções

	Notação usual (in-fixa)	Notação polonesa (pós-fixa)
1	a+b	ab+
2	a+b+c	ab+c+ ou abc++
3	a+b*c	abc*+
4	(a+b)*c	ab+c*
5	c*(a+b)	cab+*
6	b*b-4*a*c	bb*4a*c*- ou bb*4ac**-
7	(-b+√(b*b-4*a*c))/(2*a)	b-bb*4a*c*-√+2a*/

Observações sobre a notação pós-fixada

- Elimina a necessidade de parêntesis.
- Pode existir mais de uma solução (exemplos 2 e 6) devido a mesma prioridade dos operandos envolvidos
  - nos 2 exemplos é melhor considerar a primeira solução, pois a convenção usual é que quando temos operadores de mesma prioridade (no exemplo 2 ++ e no exemplo 6 **) as operações são feitas da esquerda para a direita.
- Os operandos aparecem na mesma ordem em que se encontram na expressão original.
- Operadores diferentes que usam o mesmo símbolo podem causar confusão na notação pós-fixada
  - Operadores unários – e + que podem ser interpretados como a subtração e adição normais.
  - Potenciação (**) que pode ser interpretada como multiplicações sucessivas.
  - É conveniente, ao traduzir para a notação pós-fixa, usar outro símbolo para distinguir os operadores.

Observações sobre a notação pós-fixa

- Para calcular o valor da expressão em notação pós-fixada: varre-se a expressão da esquerda para a direita
  - As operações são realizadas à medida que aparecem.
  - Não é necessário verificar qual a operação a ser feita primeiro devido a sua prioridade ou a existência de parêntesis.
  - Decorrência: se houver uma maneira fácil de traduzir uma expressão para a sua correspondente forma pós-fixada, o cálculo da mesma será feito de forma direta, simplesmente percorrendo a expressão da esquerda para a direita.

Transformação para notação pós-fixa (Algoritmo)

- Varre-se a expressão in-fixa da esquerda para a direita:
  - deve-se levar em conta a prioridade dos operadores antes de colocá-los na expressão pós-fixa
  - antes de colocar um operador na expressão pós-fixa, é necessário saber se o próximo operador é menos prioritário que ele
- Utiliza-se uma pilha para guardar os operadores
  - quando chega um operador mais prioritário que o do topo da pilha, empilha-se este também
  - quando chega um operador de menor ou igual prioridade, o do topo tem que ir para a expressão pós-fixa e dar lugar para este que acabou de chegar
- Caso especial: os parêntesis
  - quando aparece um “fecha” - deve-se colocar na pós-fixa todos os operadores até o “abre” correspondente

Transformação para notação pós-fixa (Algoritmo)

```
while (expressão não chegou ao fim):
    p = próximo elemento da expressão (operador ou operando)
    if (p é operando): coloque na pós-fixa
    if (p é operador):
      tire da pilha e coloque na pós-fixa todos os operadores
      com prioridade maior ou igual a p, na mesma ordem de
      retirada da pilha
      empilhe p;
    if (p é abre parêntesis): empilha p;
    if (p é fecha parêntesis):
      desempilhe os operadores até o primeiro abre e coloque
      na pós-fixa na mesma ordem de retirada da pilha;

desempilhe todos os operadores que ainda estão na pilha e coloque na
pós-fixa na mesma ordem de retirada da pilha
```

Transformação para notação pós-fixa (Algoritmo)

- Os operadores devem então estar organizados por sua prioridade
- Para facilitar o algoritmo: abre, fecha parêntesis também tem sua prioridade definida (tratamento especial), além das prioridades dos operandos

```
def prioridade(x):
    if x == '+': return 1
    elif x == '-': return 1
    elif x == '*': return 2
    elif x == '/': return 2
    elif x == '^': return 3
    elif x == '(': return 4 # caso particular
    elif x == ')': return 5 # caso particular
    else: return 0          # não é operador
```

Comportamento da pilha

a+b

	Pilha	Pós-fixa
a		a
+	+	a
b	+	ab
		ab+

a+b+c

	Pilha	Pós-fixa
a		a
+	+	a
b	+	ab
+	+	ab+
c	+	ab+c
		ab+c+

a*b+c

	Pilha	Pós-fixa
a		a
*	*	a
b	*	ab
+	+	ab*
c	+	ab*c
		ab*c+

a+b*c

	Pilha	Pós-fixa
a		a
+	+	a
b	+	ab
*	+*	ab
c	+*	abc
		abc*+

a*(b+c)

	Pilha	Pós-fixa
a		a
*	*	a
(	*(	a
b	*(	ab
+	*(+	ab
c	*(+	abc
)	*	abc+
		abc+*

Comportamento da pilha

a/(b*c)*d

	Pilha	Pós-fixa
a		a
/	/	a
(	/(	a
b	/(	ab
*	/(*	ab
c	/(*	abc
)	/	abc*
*	*	abc*/
d	*	abc*/d
		abc*/d*

(a+b)*c

	Pilha	Pós-fixa
(	(	
a	a	a
+	(+	a
b	(+	ab
)		ab+
*	*	ab+
c	*	ab+c
		ab+c*

a*(b+(c*(d+e)))

	Pilha	Pós-fixa
a		a
*	*	a
(	*(	a
b	*(	ab
+	*(+	ab
(	*(+(	ab
c	*(+(	abc
*	*(+(*	abc
(	*(+(*(	abc
d	*(+(*(	abcd
+	*(+(*(+	abcd
e	*(+(*(+	abcde
)	*(+(*	abcde+
)	*(+	abcde+*
)	*	abcde+*+
		abcde+*+*

Prioridade de operadores (Python)

Operator	Description
`()`	Parentheses (grouping)
`f``(args...)`	Function call
`x``[index:index]`	Slicing
`x``[index]`	Subscription
`x.attribute`	Attribute reference
`**`	Exponentiation
`~`*`x`*	Bitwise not
`+`*`x`*`, -`*`x`*	Positive, negative
`*, /, %, //`	Multiplication, division, remainder
`+, -`	Addition, subtraction
`<<, >>`	Bitwise shifts
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`in, not in, is, is not, <, <=, >, >=, <>, !=, ==`	Comparisons, membership, identity
`not` *`x`*	Boolean NOT
`And`	Boolean AND
`Or`	Boolean OR
`lambda`	Lambda expression

Prioridade de operadores (Python)

1. a > b or c < d

   a b > c d < or

2. a == b or c + d >= e * f and g

   a b == c d + e f * >= g and or

3. a > b or b < c and c == d

   a b > b c < c d == and or

4. b + c * d != x and not y > 0

   b c d * + x != y 0 > not and

Cálculo do valor de uma expressão pós-fixa

- Basta varrê-la da esquerda para a direita e efetuar as operações com os dois últimos operandos (ou o último no caso de operadores unários).
- Os operandos têm que ser empilhados à medida que aparecem, pois, a operação será aplicada aos dois últimos (se for operação binária) ou apenas ao último se for uma operação unária.

Cálculo do valor de uma expressão pós-fixa

- O resultado da expressão estará no topo da pilha (único elemento final).
- Observe que no algoritmo de tradução, a pilha era de operadores enquanto que no algoritmo de cálculo a pilha é de operandos.

```
while (expressão pós-fixa não chegou ao fim):
    seja p o próximo elemento da expressão
    if (p é operando): empilha p;
    if (p é operador unário):
        faz a operação com o elemento do topo da pilha
    if (p é operador binário):
        faz a operação com os 2 elementos do topo da pilha
        neste caso a pilha diminui de 1 elemento
```

Exemplo

a * (b + c * (d + e)) === a b c d e + * + * (supor a=1,b=2,c=3,d=4,e=5)

a	b	c	d	e	+	*	+	*
				5				
			4	4	9			
		3	3	3	3	27		
	2	2	2	2	2	2	29	
1	1	1	1	1	1	1	1	29



Obrigado!

Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr